

Name: _____

Collaborators: _____

THAYER SCHOOL OF ENGINEERING • DARTMOUTH

ENGS 28, W26

LAB 4 — ANALOG TO DIGITAL CONVERSION

Due: Tuesday February 3, 2026 2:15 PM

Contents

1 Overview	3
1.1 Place in course	3
1.2 Learning Objectives	3
1.3 Equipment List	3
2 Technical Study - ADC	3
3 Technical Study - Range Conversion	5
3.1 Conversion using Floats	5
3.1.1 Conversion	5
3.1.2 Timing	6
3.2 Conversion with Integers	7
3.2.1 Conversion	7
3.2.2 Timing	7
4 Technical Study - Temperature Sensor	8
4.1 Expectations	8
4.2 Using the temperature sensor	9
4.3 Final Touches	10
4.4 Going Further (optional)	10
5 Postlab	10
A How to print floats	11

Deliverables

1 Change Channels (1 point)	4
2 ADC Modes (2 points)	4
3 Changes required to switch modes (1 point)	5
4 Floating point (2 points)	6
5 Floating Point Timing (2 points)	6
6 Integer Range (1 point)	7
7 Integer Timing (1 point)	7
8 Voltage Range (2 point)	8

9	Temperature Range (1 point)	8
10	Voltage Output (2 point)	8
11	Prediction (1 point)	9
12	Temperature Conversion Function (1 point)	9
13	Temperature Conversion Code (1 point)	10
14	Demonstration (1 point)	10

1 Overview

1.1 Place in course

At this point in the course, you have become quite familiar with Digital input and output. We have begun to use the builtin ADC. This lab will solidify your understanding of the ADC and it's uses.

1.2 Learning Objectives

- Understand basic characteristics of analog-to-digital conversion.
- Manage calculations with fixed point arithmetic.
- Interface an analog sensor with a microcontroller.
- Continue to practice using datasheets.

1.3 Equipment List

- Nucleo / breadboard setup
- AD2 and, optionally, the digital multi-meter
- $10K\Omega$ potentiometer
- TMP235 temperature sensor
- Connector and wires for TMP235

2 Technical Study - ADC

In class, you wired a potentiometer to the analog to digital converter and used the supplied program `ADCPot.c` to observe analog to digital conversion. In this first section of the lab, you will explore different ways the on-board ADC can be configured. The purpose of this exercise is to cultivate an understanding of the relationship between the ADC input voltage and the binary ADC output.

If it isn't already setup, configure your Nucleo and potentiometer as shown in Figure 1.

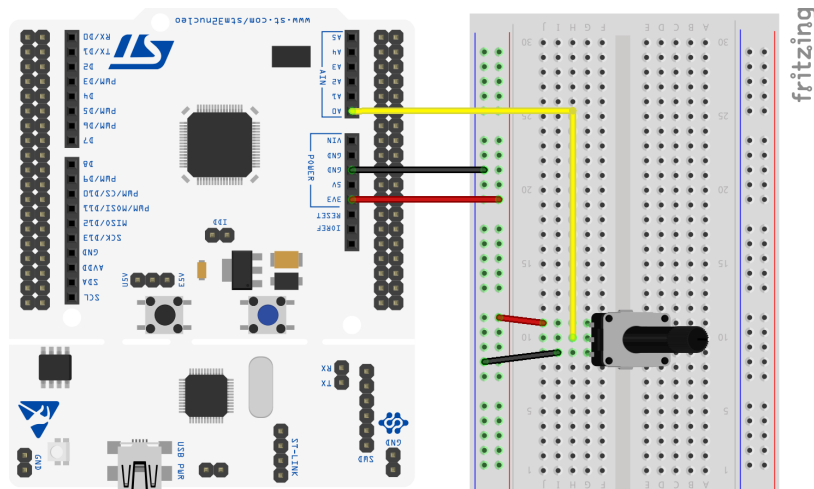


Figure 1: Breadboard Setup

Now, load the `ADCPot.c` program onto your Nucleo and open the serial monitor on your computer.

The potentiometer, a type of variable resistor, functions in this circuit as a variable voltage divider (recall the supplemental material for Lab 1). Make sure that as you turn the potentiometer knob clockwise, the ADC output values increase. (If they decrease, it means the outer terminals of the pot are wired backwards and should be swapped). Wire a voltmeter into your circuit so you can simultaneously monitor the voltage going into the ADC.

- If you use the voltmeter function of Waveforms, wire the AD2's 1+ and 1- (Orange) leads to the center terminal of the pot and to ground, respectively.
- If you use the DMM (Digital Multimeter), note that we have some jumpers wires with alligator clips at one end and a breadboard pins at the other end. Use a pair of these to connect the tip of the + lead (red) to the center terminal of the pot, and the tip of the - lead (black) to ground. Set the DMM to an appropriate DC voltage range.

Modify the `ADCPot.c` program in a few ways (make one change at a time, then change it back. You do not need to submit your full program listings for this). Keep the STM32C0x1¹ data sheet for reference—you will want to look at the datasheet early and often when solving these problems.

Change the analog input channel from 0 to 3 (rewire the circuit appropriately). Make sure that the program still works properly. In the space below, describe the change(s) you needed to make to set a particular channel. Afterward, change the input back to channel 0.

Note!

Here, Channel 3 refers to the pin labelled A3 on the board. You will need to find out what GPIO pin that is connected to (from the pinout chart on Canvas), then find out which internal ADC channel that GPIO is connected too. Lots of translations - proceed methodically!

Deliverable 1: Change Channels (1 point)

The ADC on the STM32C0x1 can be configured to report the conversion result as a 6, 8, 10 or 12 bit value. In this section, you will explore these settings by filling in the table below. For each row of the table, configure the ADC as described by the first column. Set your potentiometer output voltage to 1 volt (measured on your DMM). Calculate ΔV for the row. Calculate what the ADC should read. Compare your calculated (predicted) value to the measured value by running your code on the Nucleo and record the reading in the column "Actual". Use the space below the table to describe the changes needed for each configuration.

Deliverable 2: ADC Modes (2 points)

¹This is the STM32C0x1Reference Manual.pdf

ADC Width	ΔV	Predicted	Actual
12-bit			
10-bit			
8-bit			
6-bit			

Deliverable 3: Changes required to switch modes (1 point)

Now rotate the potentiometer through its entire range. You may notice that the lowest value you can read is not zero. You should also notice that the ADC value reaches 4095 (the max value) well before the potentiometer has reached the end of its travel. This is because there is a *DC offset* in the conversion. The amount of offset changes with temperature. Fortunately, our ADC provides an automated way to minimize the effects of this offset.

Modify your initialization routine to include the following. Place this code immediately after the access to `RCC->APBENR2`.

```
// Turn on the reference voltage
ADC1->CR |= ADC_CR_ADVREGEN;

// wait for the specified startup time (2 ms is plenty)
delay_ms(2);

// Now poke the calibrate bit and wait for it to finish
ADC1->CR |= ADC_CR_ADCAL;
while ( ADC1->CR & ADC_CR_ADCAL );
```

When this code completes, there will be a non-zero value in the `ADC1->ADC_CALFACT` register. This is used by the ADC to compensate (remove) the DC offset. Re-run your code again and observe both ends of the range to confirm that the lowest value is now (closer to) zero, and the “dead zone at” the upper end is smaller.

3 Technical Study - Range Conversion

3.1 Conversion using Floats

3.1.1 Conversion

Now we are going to take a look at a few ways to convert from the native range of the ADC ($0 \rightarrow 4095$ in 12b mode) to the real world value the sensor is representing.

As described earlier, the ADC maps the lowest voltage available (0V) to 0 and it maps the highest voltage (3.3V) to 4095. There is a linear relationship between the two extremes, so

```
float voltage = 3.3 * adc_read() / 4096;
```

is the most obvious way to do the conversion. There is one caveat however. The chip we are using (the STM32C031xx) does not have a floating point unit. This means that there is no hardware support for doing calculations using real (**float** or **double**) numbers. Any time we use a **float**, the compiler must emulate the behavior using only integer arithmetic. While possible, this is a bit costly.

Make a copy of your `ADCPot.c` (the one from the previous section with the calibration in it). Modify it to include the line above and print the floating point result.

Note!

Floating point printing (`printf( "%f\n\r", voltage);`) is not enabled by default. Refer to Appendix A for details about how to enable it.

Deliverable 4: Floating point (2 points)

Note your min and max values.

3.1.2 Timing

Now, modify the code so that rather than printing the result, it is going to time how long the code takes to run.

We have provided a function for you called `milliseconds()` which returns (as a `uint32_t`) the number of milliseconds that have elapsed since the chip was turned on. You can use this to estimate how long something takes. In our case, we are going to time how long it takes to read and convert the ADC value 10,000 times. The pseudo-code below should help get you started.

```
uint32_t a, b;

// Initialization stuff
...

// Do something we want to time
a = milliseconds();

for ( int i = 0 ; i < 10000; i++ ) {
    // read and convert (but do not print) the ADC value
}

b = milliseconds();
printf ( "10000 iterations took %d ms\r\n", b - a );
```

Note, we are explicitly *not printing* the result. Printing actually takes quite a bit of time and would pollute the timing results.

Run your code and make note of how long it takes.

Deliverable 5: Floating Point Timing (2 points)

How long did your code take to run?

3.2 Conversion with Integers

3.2.1 Conversion

The processor we are using is a native 32 bit microprocessor. This means that it has hardware specifically designed to perform arithmetic on 32 bit integers.

Replace the floating point conversion from the previous code to one using integers only. In this case, we have to make a choice about how to represent our largest value. We could round it to the nearest integer (3), but that wouldn't be very satisfying (or accurate). Another option is to represent the value in millivolts. Modify your code from above to do the conversion to millivolts. You'll also need to modify the `printf` to match the new `uint16_t` value.

```
uint16_t millivolts = 3300 * adc_read() / 4096;
```

Rerun the code to confirm that you are seeing a range that runs from 0 to 3300 millivolts.

Deliverable 6: Integer Range (1 point)

Note your min and max values.

Note, we chose to use a `uint16_t` because it is the smallest integer size that can hold our full expected range. Embedded programmers need to be very careful about data sizes because memory is quite limited on these processors. We also, however, need to be aware of the full range of our calculations. To illustrate this point, quickly modify your code to do the calculation one step at a time.

```
uint16_t millivolts = adc_read();
millivolts = millivolts * 3300;
millivolts = millivolts / 4096;
```

This appears to perform the same calculation as the first version. Run the code and see what happens.

What is going on here?

Notice that the first assignment to `millivolts` is fine. We can store a value between 0 and 65,535 in the variable and the ADC is reporting a value between 0 and 4095, so it will always fit.

What happens next? When we multiply by 3300, we get a result between 0 and 13,513,500. Most of that range exceeds the range of the variable. The hardware simply stores the lower 16 bits and throws away the rest.

The final insult occurs when we divide by 4096 which effectively throws away the lower 12 bits of what we have left. By the time we are done, we only have 4 bits of calculated value left. They come from the middle of the number and are essentially useless to us.

So why did the first version work? The C standard is very specific about how arithmetic is done. In particular, it specifies something called *usual arithmetic conversions*. In short this means that any time the compiler runs into an equation using an integer type that is smaller than the *native* type (32b for us), it converts everything to the native size before doing the calculation. So for us (in the working example), the right hand side (RHS) is calculated as a 32b value. It is only on assignment to the 16b variable on the left hand side (LHS) that it gets converted back to 16b. So for us, even though the intermediate results can overflow, the final result fits into 16b.

3.2.2 Timing

Repeat the timing portion of the experiment with your functional millivolt based code. Make sure that you are not timing the `printf`!

Deliverable 7: Integer Timing (1 point)

How long did this code take to run?
Briefly, what can you conclude about the use of floating point on this processor?

4 Technical Study - Temperature Sensor

4.1 Expectations

Open the datasheet for the TMP235 temperature sensor. Here is a set of study questions to guide you to the most important basic information about the device.

Deliverable 8: Voltage Range (2 point)

What is the recommended range of supply voltage, V_S ? What is the supply current (how much current is required by the device)? If we operate the device using the Nucleo's 5V supply, how much power does the TMP235 draw (calculate)?

Supply Range: _____
Supply Current: _____
Power Consumed: _____

Deliverable 9: Temperature Range (1 point)

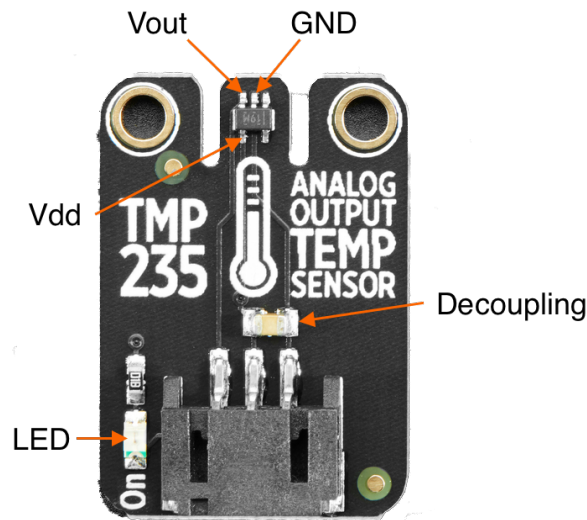
What is the range of temperatures this sensor can reliably measure?
Temp Range: _____

Deliverable 10: Voltage Output (2 point)

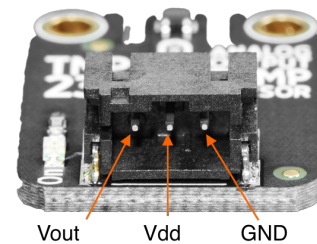
What voltage does the sensor output at 25°C: _____ ?
If the temperature increases by 1°C, by what amount does the output voltage change?
 $\Delta T/1^\circ\text{C}$: _____
Work out an equation relating the output voltage to the input temperature (you will need this later).
 $V_{out}(T)$: _____

4.2 Using the temperature sensor

You may have noticed while reading the datasheet that the TMP235 is packaged exclusively as a surface mount device (SMD). Since we cannot place any SMD components on a breadboard, we are using a small module (circuit board) which contains the TMP235, and some decoupling capacitance. It also uses a small connector for easy access to the IO of the device (V_{DD} , GND , and V_{out}). Details of this module can be found in Figure 2a and Figure 2b. The connector is keyed, which means it can only be inserted in one orientation. In our case, the Red wire connects to $+5V$, the Black wire connects to GND , and the White wire is the analog signal.



(a) TMP235 Module



(b) TMP235 Connector

Figure 2: Bipolar Schematic Symbols

Unplug the USB connector to your Nucleo to remove power from the circuit. Leave the potentiometer on your breadboard but disconnect it from the ADC.

Insert the small connector into the TMP235 board. Connect power and ground to your breadboard and connect the signal to an analog input.

Before making a measurement with your new system, predict what you expect to see. Assuming your environment is $23^{\circ}C$, calculate what the sensor's output voltage should be, and what the corresponding ADC output value should be, using $V_{REF} = 3.3\text{ V}$.

Deliverable 11: Prediction (1 point)

Output voltage $V(23)$: _____
 ADC Value: _____

Power up the system and measure the sensor's output voltage with your multimeter and convert that to a temperature with your linear equation.

Deliverable 12: Temperature Conversion Function (1 point)

What voltage did you measure?: _____
 What temp is this?: _____

Derive a simple formula to compute temperature in Celsius from the ADC value and write a program to implement it on the microcontroller. Follow the same guidelines for doing arithmetic that were demonstrated in the previous section, using the potentiometer. Because temperature can go negative, be sure to perform your calculations with `int16_t` and `int32_t` rather than `uint16_t` and `uint32_t`.

4.3 Final Touches

Earlier in this lab, we looked at how to switch channels on the ADC.

Leave the temperature sensor connected and re-connect your potentiometer to another channel. Modify your main loop to take samples from each of these channels. You should take the samples one at a time. In other words, enable one channel, take a sample, then disable the channel. Then enable the other channel, take sample, and disable it.

Print both the converted voltage (in millivolts!) from the potentiometer and the temperature from the sensor.

Take your time with this as it forms the basis for next week's lab!

Deliverable 13: Temperature Conversion Code (*1 point*)

Submit your well commented code along with this sheet. Remember - submit the raw `.c` file, not a pdf of it!

Deliverable 14: Demonstration (*1 point*)

Demonstrate your design to a TA or instructor.

Signature _____

4.4 Going Further (optional)

Derive another equation to calculate the temperature in Fahrenheit and display on the screen. You can display them side-by-side; for extra challenge, use the `C` and `F` keys on the keyboard to control the display mode.

5 Postlab

Submit the deliverables as a single PDF to Canvas.

A How to print floats

If you tried to use the

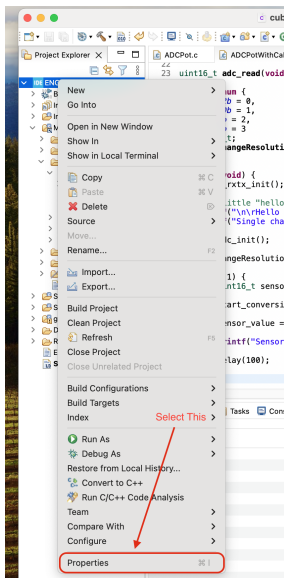
```
printf( "Value: %f", floatVar );
```

construct earlier in the lab, you likely ran into a pretty obscure message.

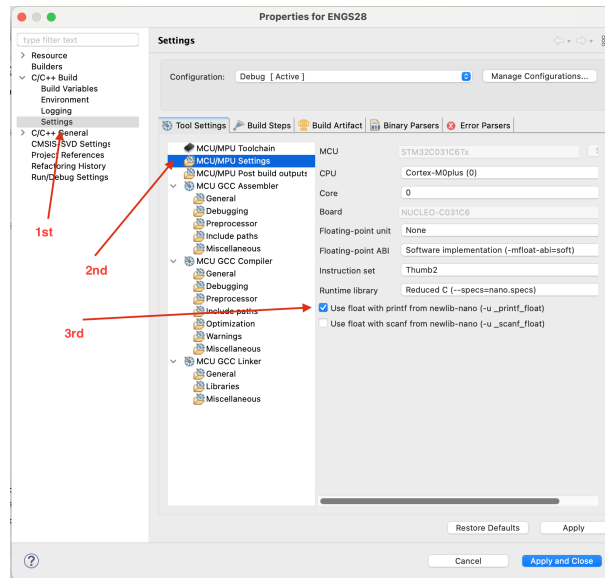
By default, the code required to allow printing of floating point numbers is not included in our standard library. If you need to print floats (think long and hard before deciding you need to print them), you can follow these instructions to enable them.

1. Open the project preferences: Right click on the project and select preferences from the bottom of the popup menu. (Figure 3a)
2. One the main preferences page, select Settings (Figure 3b)
3. In the settings pane, select MCU/MPU Settings
4. Click the **Use float with printf ...** checkbox.
5. Apply and close.

Once you rebuild, you should be able to print floating point numbers.



(a) Project PopUp Menu



(b) Main Setting Dialog